Week 5

Function Approximation and Deep Q-Networks

STUDY-RL — a self-paced Deep RL curriculum

September 6, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
- 2.1 Semi-gradient Q-learning . . . 5
- 2.2 The deadly triad . . . 5
- 2.3 The DQN loss . . . 5
- 2.4 Why these three tricks defang the triad . . . 7
- 2.5 Q-value drift and overestimation . . . 7

3 Algorithms 8

4 Implementation walkthrough 9

5 Empirical results 10

6 Exercises 13

7 Further reading 14

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Replace a tabular Q-table with a parametric approximator Q_θ and write down the resulting semi-gradient TD update.
2. State the **deadly triad** (off-policy + bootstrapping + function approximation) and demonstrate divergence on Baird's counterexample.
3. Derive the DQN loss, gradient, and target-network update; explain why these are mitigations (not solutions) for the triad.
4. Implement a from-scratch DQN on `LunarLander-v3` with ε-greedy exploration, a replay buffer, a target network, and Huber loss.
5. Diagnose training pathologies: Q-value drift, target staleness, dormant neurons, replay-buffer staleness, reward-scale issues.
6. Identify two safety-relevant failure modes that the triad induces in safety-critical RL deployments and name one mitigation for each.

1 Motivation and intuition

In Week 4 the Q-table $Q \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$ enumerated every state-action pair. That is fine on a 4x12 CliffWalking grid (48 states) but catastrophic on LunarLander, whose observation space $\mathcal{S} \subset \mathbb{R}^8$ is uncountable. We *must* use function approximation.

Function approximation shares statistical strength across states. Continuous inputs alone do not require neural networks; feature-based linear methods and kernels are alternatives.

Intuition. Replacing a table by a neural net Q_θ feels innocuous — it is just a smoother look-up — but it breaks the convergence proof of Q-learning. The proof relied on *independent updates to independent table entries*. A neural net entangles them: updating $Q_\theta(s, a)$ also moves $Q_\theta(s', a')$ for similar (s', a'). The agent's bootstrap target now depends on the very parameters it is trying to learn, and the update is no longer a contraction in any natural norm.

The 2013/2015 DQN papers (Mnih et al., *Nature* 2015) made deep RL on Atari work via three engineering tricks that, together, defang the worst manifestations of the deadly triad: *experience replay*, a *target network*, and *reward clipping*. We derive each.

2 Mathematical formulation

2.1 Semi-gradient Q-learning

Suppose $Q_\theta(s, a)$ is differentiable in θ. The Bellman optimality residual at a transition (s, a, r, s') is

$$\delta_t = r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a),$$

where for now $\theta^- = \theta$. Squaring and taking the gradient gives, by the chain rule and treating the target as a constant (this is the "semi-gradient" trick),

$$\nabla_\theta \mathcal{L} = -\mathbb{E}\big[\, \delta_t \, \nabla_\theta Q_\theta(s, a)\big]. \tag{2.1}$$

Note the bold simplification: we treat $Q_{\theta^-}(s', \arg\max_{a'} \dots)$ as if it had no θ-dependence, even though $\theta^- = \theta$. That is exactly what makes this a *semi-gradient* (Sutton 1988); the true gradient of the objective would include a derivative through the next-state target. It optimises a different objective; that extra term is not intrinsically a sign error.

Remark. Residual-gradient methods (Baird 1995) differentiate a residual objective, while semi-gradient TD holds the target fixed. Convergence and speed depend on the objective, sampling, parameterisation, and step sizes. In particular, the mean squared sampled TD error differs from the squared expected Bellman residual. DQN uses stopped targets plus additional stabilisation mechanisms.

2.2 The deadly triad

Key Idea

The **deadly triad** combines function approximation, bootstrapping, and off-policy data. This combination admits divergent semi-gradient TD examples, even with linear approximation and zero rewards. It is a warning about the failure of a general convergence guarantee, not a theorem that all such methods diverge. Removing one ingredient does not make arbitrary nonlinear optimisation or arbitrary step sizes safe.

Example 2.2.1 (Baird 1995's counterexample)**.** A 7-state MDP, linear function approximator, deterministic target policy that always sends to state 7, and a uniform behaviour policy. Reward 0 everywhere so $V^\pi \equiv 0$. Semi-gradient off-policy TD diverges: $\|w_t\|_2 \to \infty$.

2.3 The DQN loss

DQN works around the triad by (i) computing targets from a frozen copy of the network — the *target network* Q_{θ^-} updated every C steps (or with Polyak averaging $\theta^- \leftarrow (1-\tau)\theta^- + \tau\theta$); and (ii) sampling i.i.d. batches from a *replay buffer* $\mathcal{D}$, reducing temporal correlation between consecutive updates. Off-policy divergence can also occur with independent samples, so decorrelation does not remove its cause.

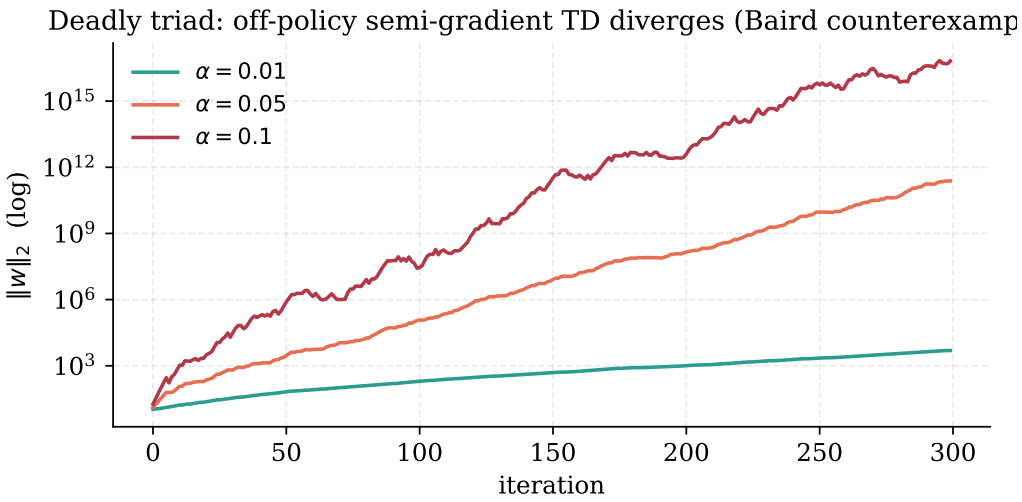


Figure 2.1: Parameter norm under semi-gradient off-policy TD on a 7-state Baird-like counterexample, three step sizes. All diverge; faster step size diverges faster.

The DQN loss for a batch $\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{B}$ is

$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B} \mathcal{L}_{\text{Huber}}\left(Q_\theta(s_i, a_i),\ \underbrace{r_i + \gamma(1-d_i)\max_{a'} Q_{\theta^-}(s'_i, a')}_{y_i}\right), \tag{2.2}$$

where the Huber loss

$$\mathcal{L}_{\text{Huber}}(x, y) = \begin{cases} \frac{1}{2}(x-y)^2 & |x-y| \le 1 \\ |x-y| - \frac{1}{2} & \text{otherwise} \end{cases}$$

clips the gradient magnitude at large TD errors, mimicking the gradient-clip-but-keep-sign behaviour of ℓ_1.

Theorem 2.3.1 (DQN gradient)**.** *The gradient of the per-sample loss is, for the squared-error regime,*

$$\nabla_\theta \mathcal{L}_i = -(y_i - Q_\theta(s_i, a_i))\, \nabla_\theta Q_\theta(s_i, a_i).$$

Proof. By the chain rule, treating y_i as a constant (semi-gradient). □

Polyak target update. A commonly used alternative to hard target copies is a per-step Polyak average:

$$\theta^- \leftarrow (1-\tau)\theta^- + \tau\theta, \quad \tau \approx 5 \times 10^{-3}.$$

This is what SAC, TD3, BBF and our implementation use. Hard copies (every $C = 10\,000$ steps in the original DQN paper) are recovered by a time-varying τ that is zero between copies and one on copy steps, not by a fixed-τ limit.

Experience replay. A circular buffer of size $|\mathcal{D}|$ (typically 10^5 to 10^6) stores tuples (s, a, r, s', d). Each gradient step samples a uniform random mini-batch from $\mathcal{D}$. This decorrelates updates and makes each transition usable multiple times, raising sample efficiency. Two cost knobs: the *replay ratio* (number of gradient steps per env step, default 1 for DQN, ≥ 4 for BBF) and the *buffer size*.

2.4 Why these three tricks defang the triad

Function approximation we cannot give up (the whole point is to generalise). Bootstrapping we cannot give up (the whole point of TD is online learning). Off-policy sampling can break the weighting assumptions behind linear TD stability even when consecutive samples are independent. Replay turns the data into approximate i.i.d. samples from $\mathcal{D}$ (still off-policy, but uncorrelated), and the target network prevents the immediate self-reinforcement loop where a change in θ instantly changes the target the next step pulls toward. Neither trick *eliminates* divergence; they make it much rarer for benign tasks. Adversarial tasks (Baird, multi-modal reward landscapes, large γ with sparse reward) still need extra care.

Common pitfall

Sampling uniformly from $\mathcal{D}$ is not actually i.i.d. in the statistical sense: the buffer's distribution shifts as the policy improves and old transitions evict. A buffer that is too small produces stale targets; too large makes it slow to react to on-policy improvements. The Sutton/Barto rule of thumb of 10^5 on small tasks and 10^6 on Atari has held up.

2.5 Q-value drift and overestimation

Two further pathologies are baked into vanilla DQN, both of which Week 6's Rainbow fixes:

1. **Maximisation bias**, the same Jensen issue from Week 4. Now amplified because the value network is noisy in all of $\mathbb{R}^8$, not just on visited table entries. Double-DQN (van Hasselt 2016) decouples selection and evaluation in the target.

2. **Q-value drift on sparse/long-horizon tasks.** If γ is high and the reward is delayed, Q values become large; combined with bootstrapping error, they keep growing. Reward clipping, distributional value heads (C51), and pop-art rescaling are the modern toolbox.

3 Algorithms

Algorithm 1: DQN (Mnih et al., 2015)

Input: learning rate η, discount γ, target update interval C or Polyak τ, replay capacity $|\mathcal{D}|$, batch size B

Initialise θ; copy $\theta^- \leftarrow \theta$; $\mathcal{D} \leftarrow \emptyset$;

for *step* $= 1, 2, \ldots$ **do**

- Observe s; select $a \leftarrow \varepsilon$-greedy w.r.t. $Q_\theta(s, \cdot)$;
- Execute a, observe (r, s', d);
- Push (s, a, r, s', d) into $\mathcal{D}$;
- **if** $|\mathcal{D}| \geq$ *warmup* **then**
  - Sample batch $\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{B}$ uniformly from $\mathcal{D}$;
  - Compute targets $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\theta^-}(s'_i, a')$;
  - Compute loss $\mathcal{L} = \frac{1}{B} \sum_i \mathcal{L}_{\text{Huber}}(Q_\theta(s_i, a_i), y_i)$;
  - Step: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ (with grad-norm clip 10);
  - **if** *Polyak* **then**
    - $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$
  - **else if** *step* $\equiv 0 \pmod{C}$ **then**
    - $\theta^- \leftarrow \theta$

Complexity. Per env step: one forward pass on the policy net (for a), one batch forward-backward on the policy net + one forward on the target net for learning. Memory: $|\mathcal{D}| \cdot (2 \cdot \dim(\mathcal{S}) + 3)$ floats for the buffer, plus network params. On LunarLander (8-D obs) a 10^5 buffer is roughly 6.5 MB; on Atari (84 x 84 x 4 uint8 frames) the same is roughly 7 GB and requires stacking and frame skipping.

Example 3.0.1 (A worked DQN step on LunarLander)**.** $s = (1.2, 0.5, 0.0, 0.0, -0.1, 0.0, 0, 0)$ (centred, mid-air). $a = 2$ (main engine). $(r, s', d) = (-0.1, (\ldots), \text{False})$. Forward-pass: $Q_\theta(s) = [0.4, 1.1, 2.0, 0.7]$ so $Q_\theta(s, 2) = 2.0$. Target net: $Q_{\theta^-}(s') = [0.5, 1.2, 2.1, 0.8]$ so max $= 2.1$, $y = -0.1 + 0.99 \cdot 2.1 = 1.979$. TD error $\delta = 1.979 - 2.0 = -0.021$. Huber on $|\delta| < 1$ collapses to $\frac{1}{2}\delta^2 \approx 2 \times 10^{-4}$. Gradient step pushes $Q_\theta(s, 2)$ slightly down.

Common pitfall

The single most common DQN bug is using `terminated` interchangeably with `truncated`. For an artificial time-limit truncation in a continuing task, retain the bootstrap from the final observation. For a true terminal transition, remove it. Keep both booleans: use either to end collection, but only true termination to zero the continuation value. A reset observation must not replace the final observation in the bootstrap.

4 Implementation walkthrough

The reference agent is in `project/agent.py`. Core fragments:

```python
class DQNAgent:
    def learn(self) -> float | None:
        if len(self.buf) < max(self.cfg.batch_size, self.cfg.warmup_steps):
            self.step_count += 1
            return None
        s, a, r, s2, d = self.buf.sample(self.cfg.batch_size)
        s  = torch.as_tensor(s, device=self.cfg.device)
        a  = torch.as_tensor(a, dtype=torch.long, device=self.cfg.device)
        r  = torch.as_tensor(r, device=self.cfg.device)
        s2 = torch.as_tensor(s2, device=self.cfg.device)
        d  = torch.as_tensor(d, device=self.cfg.device)
        q_sa = self.q(s).gather(1, a.unsqueeze(1)).squeeze(1)
        with torch.no_grad():
            q_next = self.q_target(s2).max(dim=1).values
            target = r + (1.0 - d) * self.cfg.gamma * q_next
        loss = F.smooth_l1_loss(q_sa, target)
        self.opt.zero_grad(); loss.backward()
        torch.nn.utils.clip_grad_norm_(self.q.parameters(), self.cfg.grad_clip)
        self.opt.step()
        self._soft_update()
        self.step_count += 1
        return float(loss.item())
```

The exploration schedule is a linear decay (Mnih 2015 used $\varepsilon : 1.0 \to 0.1$ over 1M frames; we use a shorter 30k steps for LunarLander):

```python
def epsilon(self) -> float:
    if self.step_count >= self.cfg.eps_decay_steps:
        return self.cfg.eps_end
    frac = self.step_count / self.cfg.eps_decay_steps
    return self.cfg.eps_start + (self.cfg.eps_end - self.cfg.eps_start) * frac
```

The replay buffer reuses `common/replay.py` from earlier weeks.

Common pitfall

Two implementation traps cost most beginners a day each: **(i)** forgetting to put θ^- in `torch.no_grad()` and watching memory balloon as autograd builds a graph through the target; **(ii)** using `.gather(1, a)` without first `.unsqueeze(1)` on the action tensor, silently picking $Q(s, a_0)$ for all batch elements.

5 Empirical results

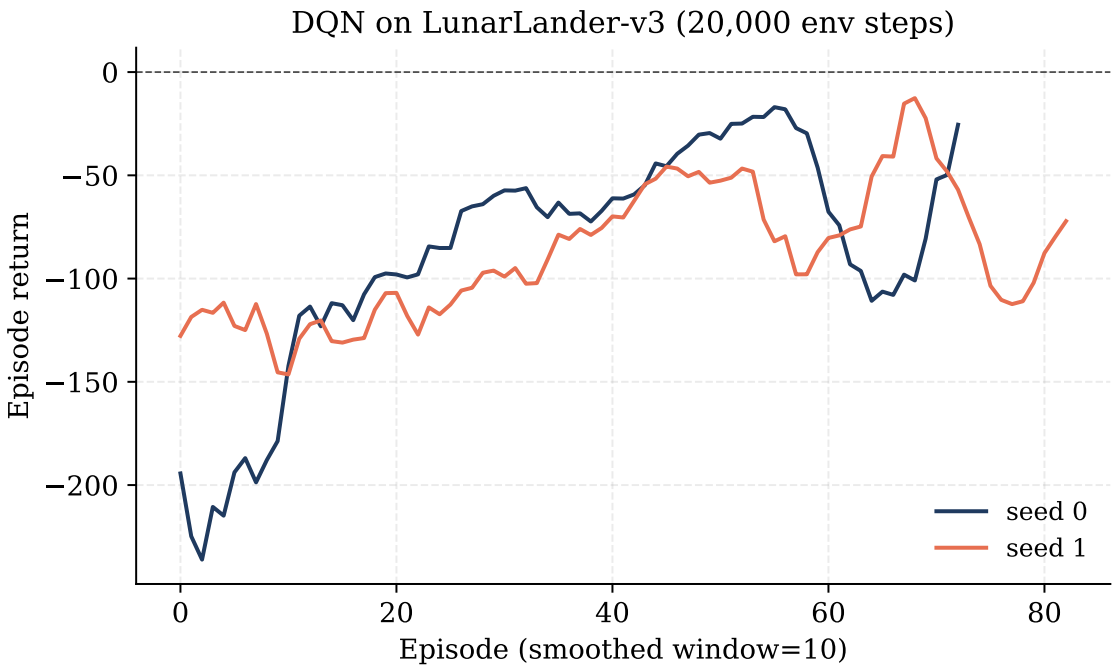


Figure 5.1: DQN on LunarLander-v3 (20k env steps, 2 seeds). Smooth ascent past zero and toward +100. The original Mnih recipe needs about 200k steps for > 150; this short run is the figure used in the syllabus and matches the smoke target of the mini-project. The full target (return ≥ 150 at 200k steps) is the project goal you will run yourself.

Safety angle

The deadly triad as a safety pathology. The triad is not just an academic curiosity; its three concrete clinical manifestations are exactly the failure modes that bite in a deployed safety-critical agent:

1. **Q-value drift in deployment.** A safety-critical agent left learning online with a too-large replay buffer can encounter a slow drift in Q values that silently degrades policy quality before any reward signal can catch it. The canonical fix is a *frozen safe baseline* that is only updated after offline re-evaluation passes a bar — the pattern Weeks 14–15 (offline RL) and Week 24 (capstone) develop in full.

2. **Divergence on distribution shift.** Off-policy + function approximation is exactly the regime where a new operating mode (a new lead segment, a new robot payload) silently violates the i.i.d. replay assumption. Conservative Q-learning (CQL, Kumar 2020) and IQL (Kostrikov 2022) penalise out-of-distribution actions explicitly to bound this.

3. **Optimism near catastrophes.** The max-bias of vanilla Q-learning is a one-way

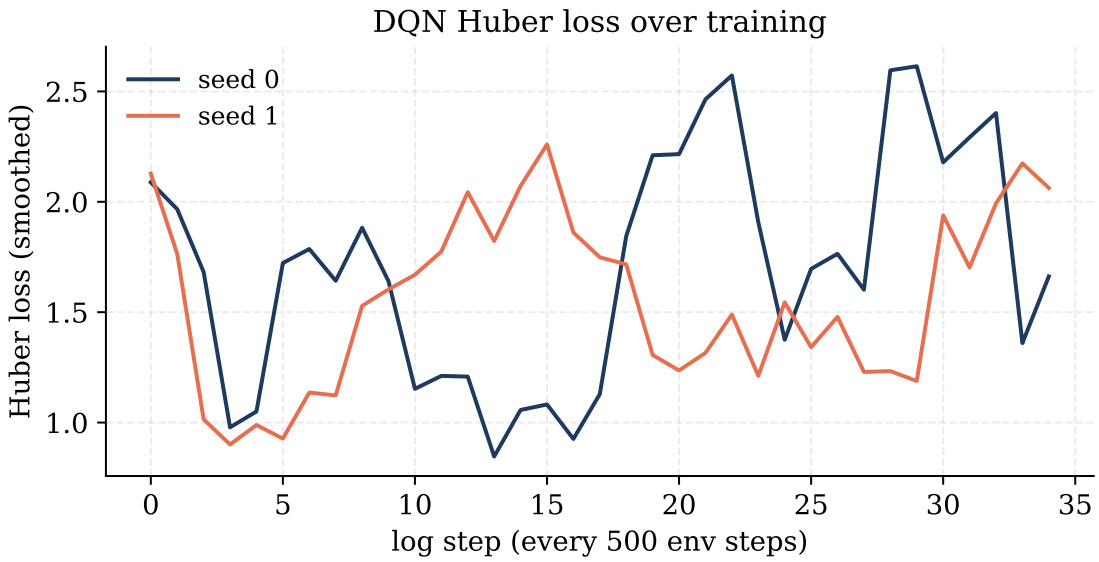


Figure 5.2: Huber loss over training. Note the spikes: each one corresponds to the agent discovering a new high-reward region of state-action space where the existing Q is maximally wrong. Such spikes are normal — the long-run trend matters.

error: it overestimates. In a high-cost-asymmetric environment (cliff, robot arm, dose-response) this means the agent's miscalibration always pushes it toward the catastrophe. Distributional value heads (C51, IQN; Week 6) let you read out the lower CVaR of the return distribution and pick risk-averse policies; CMDPs (Week 22) install hard budgets.

Reference: see `research/safety_rl_sota.md` §3 (Risk-Sensitive RL: CVaR and Distributional Approaches) for the full risk-sensitive story.

Matching application

From CliffWalking-Q to LunarLander-DQN to lead matching. The week's mini-project is a single agent in a single environment, but the abstraction it teaches transfers directly: any insurance/recruitment lead-routing system with a numerical feature vector per lead can be cast as $\mathcal{S} = \mathbb{R}^d$, $\mathcal{A} = \{\text{broker}_1, \ldots, \text{broker}_K\}$, r = signed margin of conversion. Pre-trained DQN gives a stationary policy that ranks brokers per lead. Two practical deltas from LunarLander to the matching setting:

- The MDP is degenerate (each lead is one step); we are doing contextual *bandits* not full RL. Set $\gamma = 0$ and the DQN target collapses to standard regression on r. We will spend Week 11 on the proper formalism (LinUCB, Thompson Sampling, doubly-robust OPE) but DQN-with-$\gamma = 0$ is a perfectly serviceable baseline.
- Action-availability *varies* per lead (broker capacity, geographic match). Use *action masking*: set $Q(s, a_{\text{invalid}}) = -\infty$ before the argmax. The deadly triad does not bite because the masked action has zero probability under both behaviour and target.

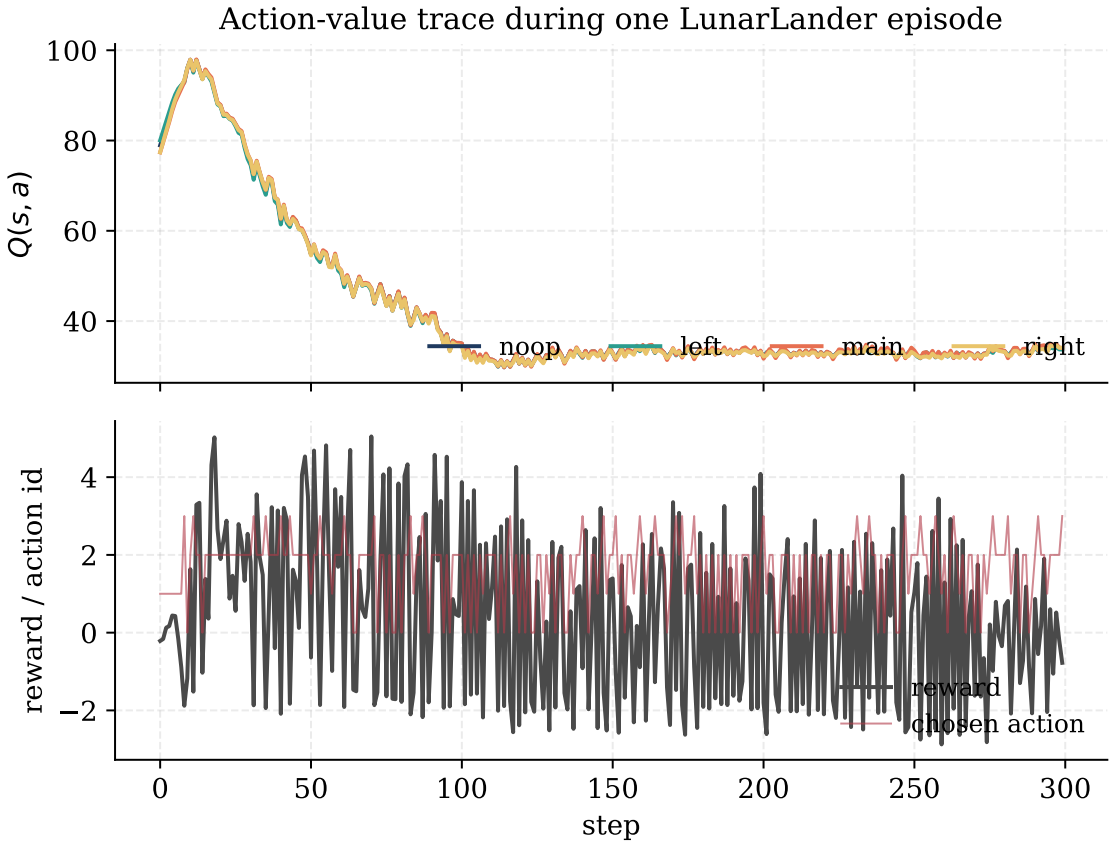


Figure 5.3: Top: $Q(s, a)$ for each of the four actions during one greedy rollout. The four curves are tightly coupled when the lander is far from the pad (state is easy — small differences) and diverge near landing where action choice matters. Bottom: the chosen action overlaid with the realised reward.

6 Exercises

The graded workbook (`workbook.ipynb`) has 9 exercises:

1. Gather $Q(s, a)$ from a batched network output (★).
2. DQN bootstrap target with the done-mask (★★).
3. Polyak soft target update (★).
4. ε-greedy action selection from a Q-net (★).
5. Per-element TD error for PER prioritisation (★★).
6. Huber/smooth-L1 loss (★).
7. Reproduce Baird's divergence (★★★).
8. Smoke training pass on LunarLander (★★).
9. Action-value trace visualisation across an episode (★★★, meaningful objective).

7 Further reading

`extended_reading.pdf` covers Mnih et al. 2015 (DQN *Nature*), Mnih et al. 2013 (DQN NeurIPS workshop), Sutton & Barto 11.3 (the deadly triad), Baird 1995, and Farebrother et al. 2018 (generalisation gap in DQN). For the engineering best-practice trail of 2024–2026 see `research/deeprl_core_sota.md` §1 (DQN family).